Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

ANHANG

D

Kontrollstrukturen in Java

D.1 Kontrollstrukturen

Kontrollstrukturen regeln die Abfolge, in der Anweisungen ausgeführt werden. Es gibt im Wesentlichen zwei Kategorien von Kontrollstrukturen: *Auswahlanweisungen* und *Schleifen*.

Eine Auswahlanweisung ist wie eine Gabelung, an der eine Entscheidung getroffen werden muss, welchem Weg durch den Rumpf der Methode (oder des Konstruktors) zu folgen ist. Eine *if-else-Anweisung* verlangt eine Entscheidung zwischen zwei Anweisungsblöcken, während eine *switch-Anweisung* mehrere alternative Anweisungsblöcke zur Auswahl anbietet.

Schleifen ermöglichen die wiederholte Ausführung eines Anweisungsblocks. Die Anzahl der Wiederholungen kann vorgegeben oder undefiniert sein. Für die erste Variante werden typischerweise *for-each*- oder *for-Schleifen* eingesetzt, die zweite Variante wird meist mit *while*- oder *do-Schleifen* realisiert.

Diese Charakterisierungen sind allerdings nicht verbindlich, Abweichungen sind in der Praxis häufig zu finden. Beispielsweise kann man die `if-else`-Anweisung auch zur Auswahl zwischen mehreren Alternativen nutzen, wenn der `else`-Teil eine untergeordnete (verschachtelte) `if-else`-Anweisung enthält, und eine `for`-Schleife kann auch für eine nicht festgelegte Anzahl von Wiederholungen eingesetzt werden.

D.2 Auswahlanweisungen

D.2.1 if-else

Die *if-else-Anweisung* gibt es in zwei Varianten, die beide durch einen booleschen Ausdruck gesteuert werden.

```
if (Ausdruck {
   Anweisungen
}
```

```
if (Ausdruck) {
   Anweisungen
}
else {
   Anweisungen
}
```

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

In der ersten Variante entscheidet der boolesche Ausdruck darüber, ob die Anweisungen ausgeführt werden oder nicht. In der zweiten Variante entscheidet der Ausdruck, welcher der beiden alternativen Anweisungsblöcke ausgeführt werden soll.

Beispiele:

```
if (feld.size() == 0) {
    System.out.println("Das Feld ist leer.");
}
if (zahl < 0) {
    meldeFehler();
}
else {
    verarbeiteZahl(zahl);
}
```

Eine in der Praxis recht häufig zu findende Konstruktion ist die Verknüpfung von **if-else**-Anweisungen, wobei eine zweite **if-else**-Anweisung in den **else**-Teil der ersten Anweisung eingefügt wird. Dieses Muster kann beliebig oft wiederholt werden, Sie sollten aber darauf achten, die Konstruktion mit einem **else**-Teil abzuschließen.

```
if (zahl < 0) {
    verarbeiteNegative(zahl);
}
else if(zahl == 0) {
    verarbeiteNull();
}
else {
    verarbeitePositive(zahl);
}
```

D.2.2 switch

Die *switch-Anweisung* verzweigt auf Basis eines einzelnen Werts in eine beliebige Anzahl von Fällen (*cases*). Zwei mögliche Programmiermuster sind:

```
switch (Ausdruck) {
    case Wert1: Anweisungen1;
                break;
    case Wert2: Anweisungen2;
                break;
    weitere Fälle hier ausgelassen
    default: Anweisungen3;
             break;
}
```

```
switch (Ausdruck) {
    case Wert1:
    case Wert2:
    case Wert3:
        Anweisungen1;
        break;
    case Wert4:
    case Wert5;
        Anweisungen2;
        break;
    weitere Fälle hier ausgelassen
    default:
        Anweisungen3;
        break;
}
```

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

Hinweise:

- Eine **switch**-Anweisung kann beliebig viele **case**-Klauseln enthalten.
- Die **break**-Anweisung nach einer **case**-Klausel sorgt dafür, dass die Ausführung nicht in die folgende **case**-Klausel „durchläuft". Die zweite Form oben macht Gebrauch von diesem „Durchlaufen". In diesem Fall führen die ersten drei Werte zur Ausführung von *Anweisungen1*, während *Wert4* und *Wert5* zur Ausführung von *Anweisungen2* führen.
- Die **default**-Klausel, die „alle anderen Fälle" abdeckt, ist optional. Wenn keine **default**-Klausel angegeben ist, wird möglicherweise kein Fall ausgeführt.
- Die **break**-Anweisung nach der **default**-Klausel (oder nach der letzten **case**-Klausel, wenn keine **default**-Klausel angegeben ist) ist nicht notwendig, wird jedoch als guter Stil angesehen.
- Der Ausdruck, zu dem ausgewertet wird, sowie die **case**-Klauseln können vom Typ String sein.
- Der Ausdruck und die **case**-Klauseln können ein Aufzählungstyp sein. Der Typname wird bei den Klauseln weggelassen.

Beispiele:

```
switch(tag) {
    case 1: tagString = "Montag";
            break;
    case 2: tagString = "Dienstag";
            break;
    case 3: tagString = "Mittwoch";
            break;
    case 4: tagString = "Donnerstag";
            break;
    case 5: tagString = "Freitag";
            break;
    case 6: tagString = "Samstag";
            break;
    case 7: tagString = "Sonntag";
            break;
    default: tagString = "ungültiger Tag";
            break;
}
switch(dow.toLowerCase) {
    case 1 "Mon";
    case 2 "Die";
    case 3 "Mit";
    case 4 "Don";
    case 5 "Fre";
            zurArbeitGehen();
            break;
    case 6 "Sam";
    case 7 "Son";
            imBettBleiben();
            break;
}
```

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

D.3 Schleifen

Java bietet drei Schleifen: *while*, *do-while* und *for*. Die **for**-Schleife gibt es in zwei Formen. Die **while**- und die **do-while**-Scheife eignen sich besonders gut für unbegrenzte Wiederholungen. Die **for-each**-Schleife wird bei einer bestimmten Anzahl von Iterationen über eine Sammlung eingesetzt und die **for**-Schleife liegt zwischen den beiden. Außer bei der **for-each**-Schleife werden die Wiederholungen in allen Schleifen durch einen booleschen Ausdruck gesteuert.

D.3.1 while

Die *while-Schleife* führt einen Anweisungsblock wiederholt aus, solange die Auswertung der Schleifenbedingung **true** liefert. Der Ausdruck für die Schleifenbedingung wird vor der Ausführung des Schleifenrumpfes ausgewertet, sodass der Schleifenrumpf eventuell gar nicht ausgeführt wird. Dies ist eine wichtige Eigenschaft der **while**-Schleife.

```
while (Ausdruck) {
    Anweisungen
}
```

Beispiele:

```
System.out.print("Geben Sie einen Dateinamen ein: ");
eingabe = liesEingabe();
while(eingabe == null) {
   System.out.print("Bitte noch einmal versuchen: ");
   eingabe = liesEingabe();
}

int index = 0;
boolean gefunden = false;
while(!gefunden && index < liste.size()) {
   if(liste.get(index).equals(eintrag)) {
      gefunden = true;
   }
   else {
      index++;
   }
}
```

D.3.2 do-while

Die *do-while-Schleife* führt einen Anweisungsblock wiederholt aus, solange die Auswertung der Schleifenbedingung **true** liefert. Der Ausdruck für die Schleifenbedingung wird nach der Ausführung des Schleifenrumpfes ausgewertet, sodass der Schleifenrumpf mindestens einmal ausgeführt wird. Dies ist ein wichtiger Unterschied zur **while**-Schleife.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

```
do {
   Anweisungen
} while (Ausdruck)
```

Beispiel:

```
do {
   System.out.print("Geben Sie einen Dateinamen ein: ");
   eingabe = liesEingabe();
} while (eingabe == null);
```

Beispiel:

```
do {
   System.out.print("Geben Sie einen Dateinamen ein: ");
   eingabe = liesEingabe();
} while (eingabe == null);
```

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

D.3.3 for

Die *for-Schleife* hat zwei Formen. Die erste ist auch bekannt als *for-each-Schleife* und wird ausschließlich verwendet, um über die Elemente einer Sammlung zu iterieren. Der Schleifenvariablen wird in jedem Schleifendurchlauf der Wert eines Elements der Sammlung zugewiesen.

```
for (Variablendeklaration : Sammlung) {
    Anweisungen
}
```

Beispiel:

```
for(String notiz : liste) {
    System.out.println(notiz);
}
```

Für die Elemente der Sammlung steht kein Indexwert zur Verfügung. Die **for-each**-Schleife kann nicht verwendet werden, wenn die Sammlung während des Schleifendurchlaufs verändert werden soll.

Die zweite Form der **for**-Schleife wird ausgeführt, solange die Auswertung der Schleifenbedingung **true** liefert. Bevor die Schleife ausgeführt wird, wird eine *Anweisung zur Initialisierung* genau einmal ausgeführt. Die Schleifenbedingung wird vor Ausführung des Schleifenrumpfes ausgewertet (sodass die Schleife eventuell gar nicht durchlaufen wird). Eine *Inkrement-Anweisung* wird nach jeder Ausführung des Schleifenrumpfes ausgeführt.

```
for (Initialisierung; Schleifenbedingung; Inkrement) {
    Anweisungen
}
```

Beispiel:

```
for(int i = 0; i < text.size(); i++) {
   System.out.println(text.get(i));
}
```

Die **for**-Schleife (in beiden Formen) wird üblicherweise immer dann eingesetzt, wenn der Schleifenrumpf für eine bestimmte Anzahl von Wiederholungen ausgeführt werden soll – beispielsweise einmal für jedes Element in einer Sammlung – obwohl eine **for**-Schleife eigentlich näher an einer **while**-Schleife ist als an einer **for-each**-Schleife

D.4 Exceptions

Das Werfen und Fangen von Exceptions stellt eine weitere Möglichkeit dar, den Kontrollfluss zu ändern. Exceptions dienen aber vor allem dem Antizipieren und der Behandlung von Ausnahmesituationen – und nicht der normalen Kontrollflusssteuerung.

```
try {
    Anweisungen
}
catch (Exception-Typ Name) {
    Anweisungen
}
finally {
    Anweisungen
}
```

Beispiel:

```
try {
    FileWriter writer = new FileWriter("foo.txt");
    writer.write(text);
    writer.close();
}
catch(IOException e) {
    Debug.meldeFehler("Schreiben der Textdatei schlug fehl.");
    Debug.meldeFehler("Die Exception lautet: " + e);
}
```

Eine Exception-Anweisung kann eine beliebige Anzahl von *catch-Klauseln* haben. Sie werden in ihrer Reihenfolge im Quelltext ausgewertet und es wird nur die erste passende Klausel ausgeführt. (Eine Klausel passt, wenn der dynamische Typ des Exception-Objekts, das geworfen wurde, zuweisungskompatibel zum deklarierten Exception-Typ in der **catch**-Klausel ist.) Die *finally-Klausel* ist optional.

Java 7 führte zwei wichtige Ergänzungen zu der **try**-Anweisung ein: mehrere Exceptions mit einem **catch** fangen und **try**-Anweisung mit Ressourcen oder automatisches Ressourcen-Management (*ARM*).

Mehrere Exceptions können in derselben **catch**-Klausel behandelt werden, indem sie in einer Liste von Exception-Typen aufgeführt werden, die durch das **|**-Symbol getrennt sind.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

Beispiel:

```
try {
    ...
    var.TueEtwas();
    ...
}
catch(EOFException | FileNotFoundException e) {
    ...
}
```

Automatisches Ressourcen-Management – auch als „**try**-Anweisung mit Ressourcen" bezeichnet – trägt dem Umstand Rechnung, dass **try**-Anweisungen oft verwendet werden, um Anweisungen zu schützen, die auf Ressourcen angewiesen sind, die geschlossen werden sollten, wenn ihre Benutzung – erfolgreich oder nicht – beendet ist. Der Kopf der **try**-Anweisung wird erweitert um das Öffnen der Ressource (oft eine Datei) und diese Ressource wird am Ende der **try**-Anweisung automatisch geschlossen.

Beispiel:

```
try (FileWriter writer = new FileWriter(dateiname)) {
    ...
    hier wird der writer verwendet
    ...
}
catch(IOException e) {
    ...
}
```

D.5 Zusicherungen

Zusicherungen dienen hauptsächlich zur Überprüfung des Quelltextes während der Entwicklungsphase. Für Zusicherungen gibt es zwei Formen der **assert**-Anweisung:

```
assert boolescher-Ausdruck ;
assert boolescher-Ausdruck : Ausdruck ;
```

Beispiele:

```
assert gibDetails(schluessel) != null;
assert erwartet == aktuell :
       "Der aktuelle Wert: " + aktuell +
       " passt nicht zum erwarteten Wert: " + erwartet;
```

Wenn der boolesche Ausdruck einer Zusicherung **false** liefert, wird ein **AssertionError** geworfen.

Über einen Compiler-Schalter können Sie dafür sorgen, dass im ausgelieferten übersetzten Programm keine **assert**-Anweisungen vorhanden sind, ohne dass Sie diese Anweisungen aus dem Quelltext entfernen müssen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr

ANHANG

E

Java ohne BlueJ

Dieses ganze Buch hindurch haben wir BlueJ benutzt, um unsere Java-Anwendungen zu entwickeln und auszuführen. Dafür gibt es einen guten Grund: BlueJ bietet einige Werkzeuge, die typische Aufgaben bei der Entwicklung erleichtern. Insbesondere ermöglicht BlueJ die direkte Ausführung einzelner Methoden von Klassen und Objekten – das ist nützlich, um schnell ein neues Stück Quelltext zu testen.

Wir unterscheiden bei der Darstellung der Arbeit ohne BlueJ zwei Kategorien: das Ausführen einer Anwendung ohne BlueJ und das Entwickeln ohne BlueJ.

E.1 Java ohne BlueJ ausführen

Anwendungen, die an Benutzer ausgeliefert werden, werden ohne BlueJ gestartet. Sie haben üblicherweise genau einen Einstiegspunkt, an dem die Ausführung beginnt, wenn ein Benutzer die Anwendung startet.

Das konkrete Vorgehen hängt vom jeweiligen Betriebssystem ab. Üblicherweise wird mit einem Doppelklick auf ein Icon gestartet oder indem der Name der Anwendung in eine Kommandozeile eingegeben wird. Das Betriebssystem muss dann wissen, welche Methode welcher Klasse aufgerufen werden soll, um die gesamte Anwendung zu starten.

In Java ist dieses Problem über eine Konvention gelöst: Wenn ein Java-Programm gestartet wird, wird der Name der Klasse als Parameter an das Startkommando übergeben, während der Name der Methode festgelegt ist: **`main`**. Dieser Name wurde willkürlich von den Java-Entwicklern gewählt, steht jedoch fest, d.h., die Methode muss **`main`** heißen. (Der Name dieser Methode geht zurück auf die Startmethode der Programmiersprache C, von der Java ein Großteil der Syntax geerbt hat.)

Betrachten Sie beispielsweise das folgende Kommando, das in eine Kommandozeile, etwa in einem DOS-Fenster unter Windows oder in einer Unix-Shell, eingegeben wird:

```
java Spiel
```

Das Kommando **`java`** startet die virtuelle Maschine von Java. Sie ist Teil des Java Development Kit (JDK), das auf Ihrem System installiert sein muss. **`Spiel`** ist der Name der Klasse, die wir starten wollen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:52 Uhr